

Introduction à Python

Du problème au programme

Cours Python CH4 – 2026–2027

Pierre Quesson

pierre.queson@epf.fr

Qui suis-je ?



Pierre Quesson

Python dans mon parcours

Du calcul stochastique et des mathématiques financières
au traitement des données d'enquête dans mes recherches.

Enseignant-chercheur en sciences de gestion à l'EPF

Co-responsable de la Majeure

Digital Transformation & Supply Chain

2014–2017 · Télécom Paris

Ingénieur : calcul stochastique et mathématiques financières.

2018 · EHESS : théorie sociale et politique.

2019–2024 · Mines Paris – CGS

Doctorat en sciences de gestion : transformation numérique
du travail et des organisations.

À quelles questions saurez-vous répondre ?

Comment passer de « je voudrais... » à un programme ?

1. Quelles **données** faut-il et quel **résultat** attend-on ?
2. Quelles **instructions** permettent d'y arriver ?
3. Comment **tester** le résultat et comprendre une erreur ?
4. Comment **expliquer et retrouver** son travail ?
5. Comment **chercher, comprendre et justifier** une fonctionnalité nouvelle ?

Notre objectif : écrire de petits programmes que vous comprenez.

Comment transformer une question en programme ?

Combien de temps un rover peut-il rouler avec sa batterie ?

1. Simplifier la situation

Batterie : **120 Wh utilisables**.

Puissance constante : **30 W**.

Ces hypothèses définissent notre **modèle**, une simplification de la réalité.

2. Écrire le calcul

30 W : 30 Wh consommés en une heure.

Autonomie = énergie / puissance

120 Wh / 30 W = 4 heures.

Si la consommation varie, il faut adapter le calcul. Le programme applique les hypothèses que nous avons choisies.

3. Le traduire en instructions Python

Ce programme définit les données, calcule l'autonomie, puis affiche le résultat.

```
energie = 120
puissance = 30
autonomie = energie / puissance
print(autonomie)
```

Résultat affiché : 4.0

Remplacez 120 par 60 : le même programme calcule **2 heures**.

Le même calcul, plusieurs niveaux de détail

Calculer $2 + 3$: quels détails devons-nous écrire nous-mêmes ?

Code machine x86

```
10110000 00000010  
00000100 00000011
```

Les instructions du processeur, en binaire.

C

```
int resultat = 2 + 3;
```

Le calcul et un type explicite : `int` désigne un entier.

Extraits illustratifs : même calcul, mais modèles numériques différents. Ce ne sont pas des programmes complets.

Assembleur x86

```
MOV AL, 2  
ADD AL, 3
```

Des noms pour les instructions ; `AL` est un registre.

Python

```
resultat = 2 + 3
```

Le calcul, sans déclaration explicite du type.

Des langages pour des besoins différents

Au-delà de la machine : exprimer des calculs, des règles et des traitements.



En 1989, tous ces langages existent déjà. Pourquoi en créer un autre ?

Repères sélectifs, non filiation directe. Computer History Museum ; Dartmouth : BASIC ; Ritchie : C ; Stroustrup : C++ ; Python : ABC ; Perl : historique.

Pourquoi encore un langage en 1989 ?

Une tâche concrète : retrouver les erreurs dans un fichier de journal.

Un logiciel enregistre ses événements dans un fichier texte. Nous voulons lire ce fichier, repérer les lignes contenant « ERREUR » et les afficher.

Avec C : c'est possible

Il faut préciser les types et gérer les espaces mémoire utilisés pour stocker le texte.

On compile, puis on exécute le programme.

Et la lisibilité ? Des noms explicites et des blocs indentés rendent la logique visible : on peut comprendre, reprendre et adapter le programme.

Avec Python : moins de détails à gérer

Le langage fournit des opérations sur le texte et les fichiers, et gère la mémoire automatiquement.

On essaie les instructions directement, puis on les réunit dans un script.

Écrire et tester plus facilement un programme que l'on pourra reprendre.

Exemple pédagogique. C peut aussi être lisible ; Python n'est pas nécessairement plus rapide à l'exécution. G. van Rossum, préface (1996) ; FAQ Python : origines et motivations.

Python : une réponse de Guido van Rossum

1989

Début du
développement

Au CWI, aux Pays-Bas

Un outil conçu pour répondre à des besoins concrets, notamment autour du système Amoeba.

L'héritage d'ABC

Une syntaxe lisible ; l'indentation délimite les blocs d'instructions.

1991

Première version
publique

Et le nom ?

Un clin d'œil à la série humoristique *Monty Python's Flying Circus*.

Python, FAQ générale : création, ABC, première publication et origine du nom.

À quoi servait Python dans les années 1990 ?

Faire travailler ensemble des fichiers, des programmes et des machines.

Automatiser

Renommer une série de fichiers ;
lancer des tâches répétitives.

Traiter du texte

Lire des journaux d'exécution ; extraire
et compter les erreurs.

Communiquer

Écrire des outils réseau ; piloter des
services.

Prototyper et relier

Tester une idée ; appeler des
composants existants, notamment en
C.

Un « glue language » : du code qui relie des composants.

Exemples pédagogiques de tâches. G. van Rossum, préface (1996) ; Computer Programming for Everybody (1999).

Un langage devient un écosystème

Des communautés construisent des outils réutilisables : on ne repart plus de zéro.

Besoin	Outils et rôle
Calculer	NumPy : tableaux numériques ; SciPy : méthodes scientifiques.
Analyser, représenter	pandas : données en tableaux ; Matplotlib : graphiques.
Expérimenter	Jupyter : texte, code et résultats dans un même document.
Apprendre des données	scikit-learn : apprentissage automatique ; TensorFlow et PyTorch : notamment réseaux de neurones.

Dans ce cours : les fondamentaux de la programmation d'abord.

Les bibliothèques seront explorées selon les problèmes à résoudre ;
cette liste n'est pas un programme à maîtriser intégralement.

NumPy ; SciPy ; pandas ; Matplotlib ; IPython/Jupyter ; scikit-learn ; TensorFlow ; PyTorch.

Python aujourd'hui : utile et largement utilisé

2^e

sur GitHub

Par nombre de contributeurs mensuels en août 2025, derrière TypeScript.

+7 points

dans l'enquête Stack Overflow

Hausse de l'usage déclaré de Python entre les enquêtes 2024 et 2025.

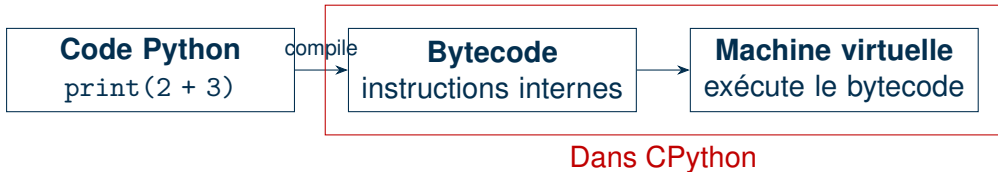
Pour quoi faire ? Automatisation, analyse de données, calcul scientifique, développement web, intelligence artificielle.

Deux indicateurs différents : ils ne donnent pas le nombre mondial d'utilisateurs de Python.

Données 2025, vérifiées le 10/09/2026. GitHub, Octoverse 2025 ; Stack Overflow, Developer Survey 2025, Technology.

Qui exécute votre code ?

Python est le langage. CPython est un logiciel qui le met en œuvre.



- Le bytecode **n'est pas** le code machine du processeur.
- CPython fonctionne sur votre ordinateur, avec le système d'exploitation.
- D'autres implémentations de Python existent, par exemple PyPy.

Schéma simplifié. Documentation Python : entrées CPython, bytecode et virtual machine.

Comment allons-nous apprendre ?

Le notebook réunit le cours, les exemples et votre travail personnel.

1. **Lisez et reformulez** les explications et la consigne.
2. **Prédisez** ce que le code va produire avant de l'exécuter.
3. **Exécutez et expliquez** : comparez le résultat à votre prédiction.
4. **Modifiez** une donnée ou une instruction ; observez ce qui change.
5. **Construisez** votre solution, puis testez-la sur plusieurs cas.

Prédire → exécuter → expliquer → modifier.

Sept séances pour progresser

Calendrier 2026 · progression ajustée selon l'avancement du groupe

	Date	Horaires	Progression
S1	16 septembre	9h–12h15	Découvrir Python.
S2	21 septembre	9h–12h15	Conditions, logique, <code>for</code> et <code>range</code> .
S3	28 septembre	9h–12h15	Contrôle 1 (1 h) , puis <code>while</code> , listes et chaînes.
S4	5 octobre	9h–12h15	Consolider et décomposer : fonctions.
S5	2 novembre*	9h–12h15	Contrôle 2 (1 h) , puis dictionnaires et fichiers.
S6	3 novembre*	13h45–17h	Problème ouvert : chercher, programmer, tester.
S7	16 novembre	9h–12h15	TP final noté (3 h) : autonomie et justification.

* Séances des 2 et 3 novembre non confirmées.

Contrôles en début de S3 et S5. Séance 7 consacrée au TP.

Récupérer le cours et conserver votre travail

Les supports du cours : github.com/Pierrotlefoot/python_intro_epf

Le notebook est aussi disponible sur **Moodle**.

1. Récupérer les supports

GitHub héberge le dépôt. Git permet d'en créer une copie locale et d'en récupérer les mises à jour.

Ouvrez le notebook de la séance dans Jupyter ; son kernel Python exécute le code.

2. Conserver vos réponses

Travaillez dans votre espace personnel, selon la procédure indiquée en cours.

Enregistrez le fichier, puis utilisez Git pour garder une version et l'envoyer vers votre dépôt distant.

Enregistrer le fichier → add → commit → push

Vérifiez ensuite vos changements sur GitHub. Un fork ne reçoit pas automatiquement les nouveautés du dépôt enseignant : suivez la procédure de synchronisation du cours.

Deux contrôles papier : comprendre sans exécuter

Une heure chacun, sans ordinateur, en début de séance.

Contrôle 1 · S3 · 28 septembre

Socle NB01 + NB02

Variables, types, conversions et calculs ;
booléens, conditions, `for`, `range`,
compteurs et accumulateurs.

Pas encore de `while`, listes ou fonctions.

Contrôle 2 · S5 · 2 novembre*

Cumulatif ; accent sur NB03 + NB04

Listes, chaînes, parcours et algorithmes ;
fonctions, paramètres et `return`.

Savoir distinguer `print` (afficher) et
`return` (renvoyer une valeur).

À travailler : prévoir un résultat, suivre les variables, repérer une erreur et écrire une solution courte.

* Séance du 2 novembre non confirmée. Coefficients et documents autorisés à préciser.

TP final : résoudre, chercher et justifier

Séance 7 · 16 novembre · TP de 3 h dans le créneau 9h–12h15.

Vous mobiliserez les fondamentaux sur un problème de traitement de données.

Certaines fonctionnalités pourront être nouvelles : vous pourrez consulter Internet et la documentation pour les découvrir.

1. **Définissez le besoin** : données, résultat attendu, étapes du traitement.
2. **Cherchez et expérimentez** : essayez la fonctionnalité sur un cas simple.
3. **Justifiez** : indiquez la ressource utilisée, expliquez le code et votre choix.
4. **Vérifiez et interprétez** : testez le résultat et expliquez ce qu'il signifie.

Les coefficients, livrables, modalités individuelles ou collectives et règles d'utilisation de l'IA seront précisés. Consulter Internet ne signifie pas que tout usage de l'IA est autorisé.

À vous : ouvrons le Notebook 01

Nous passons maintenant au cours et aux exercices dans Jupyter.

1. **Ouvrez le Notebook 01 — Introduction à Python.**

Retrouvez le fichier dans votre dossier de cours et ouvrez-le dans Jupyter.

2. **Lisez l'introduction et les objectifs.**

Repérez ce que vous allez apprendre et les consignes de travail.

3. **Commencez les premières manipulations guidées.**

Suivez les cellules dans l'ordre : lisez les explications, puis réalisez les actions demandées.

Si vous ne parvenez pas à ouvrir le notebook, signalez-le maintenant.